CONCEPTUAL SERIES
Haskell

Name: ______ Cohort/Batch: ______ Date: ______ Score: ______

HASKELL PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Programming Languages

TOTAL: 10 QUESTIONS

Q1 What are the primary design goals of Haskell? *Threat Model*

Q2 How does Haskell handle type checking: static or dynamic? *Architecture*

Q3 How does Haskell manage memory? *Configuration*

Q4 What is Haskell's concurrency model? *Hardening*

Q5 How are packages and dependencies managed in Haskell? *SSO / IdP*

Q6 How does Haskell implement object-oriented or functional programming? *Observability*

Q7 How does Haskell handle errors? *Lifecycle*

Q8 What testing tools are commonly used in Haskell? *Compliance*

Q9 What makes Haskell well-suited for its target domain? *Testing*

Q10 What are common performance pitfalls in Haskell? *Tuning*

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Haskell was designed with specific priorities

Mitigation

Haskell was designed with specific priorities around performance, safety, or developer productivity depending on its domain. These goals shape its syntax, type system, and standard library choices.

A2 Haskell uses its type system to

Primitives

Haskell uses its type system to catch errors at compile time (static) or at runtime (dynamic). This affects refactoring safety, tooling support, and the kinds of bugs that reach production.

A3 Haskell uses one of three approaches:

Hardening

Haskell uses one of three approaches: manual allocation/deallocation, a garbage collector that periodically reclaims unreachable objects, or automatic reference counting. Each has different latency and throughput tradeoffs.

A4 Haskell supports concurrency through threads,

Gotchas

Haskell supports concurrency through threads, coroutines, async/await, or an actor model. The choice determines how I/O-bound and CPU-bound workloads are scaled and how shared state is safely accessed.

A5 Haskell uses a package manager and

Federation

Haskell uses a package manager and a manifest file to declare dependencies and version constraints. A lock file pins exact versions to ensure reproducible builds across environments.

A6 Haskell supports classes and inheritance for

Observability

Haskell supports classes and inheritance for OOP, or first-class functions and immutability for FP, or both. Many modern Haskell programs blend both paradigms depending on the problem.

A7 Haskell surfaces errors via exceptions,

Automation

Haskell surfaces errors via exceptions, Result/Either types, or error return values. The choice impacts whether errors are checked at compile time and how calling code is structured.

A8 Haskell has a standard or widely

Governance

Haskell has a standard or widely adopted test runner and assertion library. Tests are typically organized into unit, integration, and end-to-end layers with coverage reporting built in or via plugins.

A9 Haskell excels in its domain due

Assessment

Haskell excels in its domain due to a combination of performance characteristics, ecosystem maturity, language ergonomics, and community support that makes common tasks simple.

A10 Typical performance issues in Haskell include

Optimization

Typical performance issues in Haskell include excessive allocations, blocking the main thread or event loop, O(n^2) data structures, and missing caching. Profiling and benchmarking tools are available to diagnose them.